Informe Final Desafío II Informática II UdeA

Curso: informática II

Desafío II: Fiesta mundial con UdeAWorldCup
Fecha de entrega: 26 de Abril de 2026

Integrantes: Frank Alexis Bedoya Toro y Daniel Elias Ayala Marichal

Análisis del problema y consideraciones para la alternativa de solución

El análisis del problema parte de la lectura rigurosa del documento guía de la clase, iniciando por la comprensión y organización del problema planteado para saber cómo dar respuesta al desafío en general, pero en especial a los requerimientos específicos que se indican de manera muy detallada en la guía, de esa misma manera propiciando la comprensión del funcionamiento real de un mundial y las dinámicas de torneo y partidos de fútbol, teniendo en cuenta el uso de la programación orientada objetos bajo la exigencia de una simulación tan demandante para la creación efectiva de un programa que responda a las necesidades de aprendizaje e implementación de conocimientos para lograr el funcionamiento adecuado.

De manera preliminar se plantea una propuesta de solución acorde a los planteamientos de la guía, indicando los requisitos para el cumplimiento de los objetivos planteados, allí se comenta lo correspondiente al proyecto y los retos técnicos que éste conllevaba, donde se plantea una contextualización, restricciones y criterios de eficiencia.

Dentro de este análisis preliminar planteamos como objetivo modelar y automatizar la simulación del mundial de la FIFA 2026 en un entorno controlado aplicando conocimientos de la programación orientada a objetos en el lenguaje de programación C++, en lo correspondiente a esto, se resuelven dudas en los encuentros sincrónicos y se toma como punto de partida el diseño preliminar de la organización del programa mediante el uso de organizadores gráficos, específicamente el UML.

Teniendo en cuenta las demandas explícitas más importantes nos encontramos con que el archivo .csv contiene 48 equipos con información importante organizada, la guía indica la necesidad de que existan 26 jugadores por equipo con nombres genéricos, la existencia de árbitros también con nombres genéricos, 12 grupos de 4 equipos, un país anfitrión (USA), unas fechas y restricciones alrededor de estás predeterminada, como que un equipo debe descansar 3 días y no pueden haber más de 4 partidos por día, para un total de 104 partidos.

Para los 48 equipos se tiene en cuenta que pertenecen a una confederación, poseen un ranking FIFA, estadísticas históricas y una plantilla de jugadores que permite estructurar el programa y dar lugar a su correcto funcionamiento. Los jugadores como entidades genéricas con estadísticas personales que se actualizan tras cada encuentro. Para los grupos se tiene en cuenta las restricciones geográficas y un sorteo de organización de mejores a peores y finalmente periodos donde transcurren todos los procesos matemáticos y operativos para

calcular probabilidades de eventos, guardar e imprimir datos mediante modelos matemáticos establecidos dando como lugar a unas estadísticas y un ganador final.

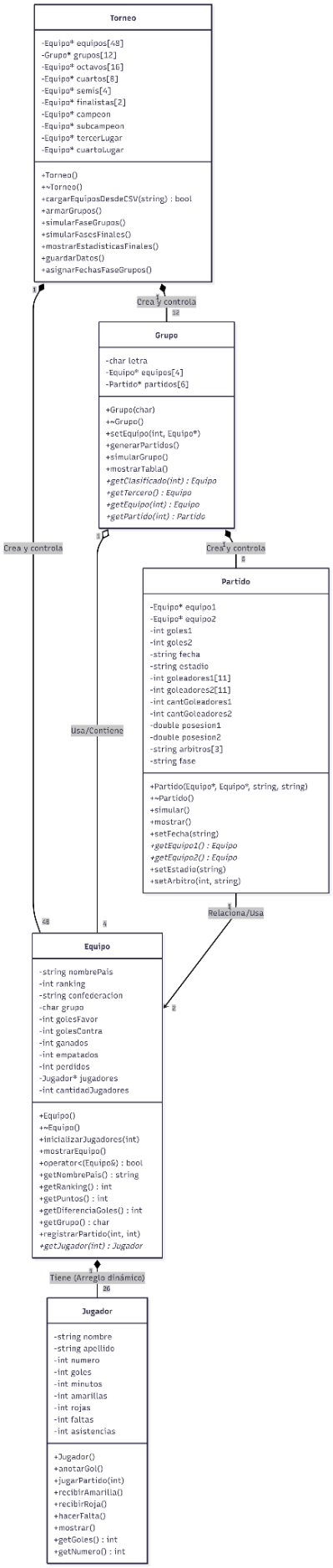
El uso de modelos matemáticos se estableció acorde a aquellos que se nos brindaron en la guía, usando la ecuación Poisson, en este caso se implementaron en ciertos puntos la librería math para la operabilidad y la aleatoriedad real dentro del programa, así mismo los porcentajes de probabilidad para ciertos eventos estaban predefinidos desde la guía base y se mantuvieron tal cual, para goles, amarillas, rojas, faltas y posesión.

Se tuvo en cuenta para las clasificatorias como hay 12 grupos, el paso a Dieciseisavos (es complejo entonces pasan los 12 primeros y los 12 segundos, solo pasan los 8 mejores terceros de comparan a los terceros de todos los grupos y aplique los criterios de desempate por total de puntos, DG, goles favor o sorteo dependiendo el caso.

Tan pronto termina un partido, se actualizan las estadísticas históricas de los jugadores y de los equipos tanto en memoria como en el almacenamiento, y al final del programa se guarda un archivo .txt con el histórico de los jugadores.

Para las restricciones no se permite el uso de librerías STL(vector, list, etc.) ni de structs,toda gestión de datos debe hacerse mediante estructuras de datos de creación propia utilizando memoria dinámica, que se implementa en todo el programa,Uso obligatorio de plantillas(templates), sobrecarga de operadores y relación entre clases, se establece para el ordenamiento descendente integrado en varias funciones y se pretende que el programa El sistema minimice el consumo de memoria y el número de instrucciones, reportando métricas de iteraciones y memoria utilizada en tiempo real.

Diagrama de clases de la solución planteada


Torneo
-Equipo* equipos[48]
-Grupo* grupos[12]
-Equipo* octavos[16]
-Equipo* cuartos[8]
-Equipo* semis[4]
-Equipo* finalistas[2]
-Equipo* campeon
-Equipo* subcampeon
-Equipo* tercerLugar
-Equipo* cuartoLugar
+Torneo()
+~Torneo()
+cargarEquiposDesdeCSV(string) : bool
+armarGrupos()
+simularFaseGrupos()
+simularFasesFinales()
+mostrarEstadisticasFinales()
+guardarDatos()
+asignarFechasFaseGrupos()
1
Crea y controla
12
Grupo
-char letra
-Equipo* equipos[4]
-Partido* partidos[6]
+Grupo(char)
+~Grupo()
+setEquipo(int, Equipo*)
+generarPartidos()
+simularGrupo()
+mostrarTabla()
+getClasificado(int) : Equipo
+getTercero() : Equipo
+getEquipo(int) : Equipo
+getPartido(int) : Partido
Crea y controla
48
1
Usa/Contiene
4
Crea y controla
6
Partido
-Equipo* equipo1
-Equipo* equipo2
-int goles1
-int goles2
-string fecha
-string estadio
-int goleadores1[11]
-int goleadores2[11]
-int cantGoleadores1
-int cantGoleadores2
-double posesion1
-double posesion2
-string arbitros[3]
-string fase
+Partido(Equipo*, Equipo*, string, string)
+~Partido()
+simular()
+mostrar()
+setFecha(string)
+getEquipo1() : Equipo
+getEquipo2() : Equipo
+setEstadio(string)
+setArbitro(int, string)
1
Relaciona/Usa
2
Equipo
-string nombrePais
-int ranking
-string confederacion
-char grupo
-int golesFavor
-int golesContra
-int ganados
-int empatados
-int perdidos
-Jugador* jugadores
-int cantidadJugadores
+Equipo()
+~Equipo()
+inicializarJugadores(int)
+mostrarEquipo()
+operator<(Equipo&) : bool
+getNombrePais() : string
+getRanking() : int
+getPuntos() : int
+getDiferenciaGoles() : int
+getGrupo() : char
+registrarPartido(int, int)
+getJugador(int) : Jugador
1
Tiene (Arreglo dinámico)
26
Jugador
-string nombre
-string apellido
-int numero
-int goles
-int minutos
-int amarillas
-int rojas
-int faltas
-int asistencias
+Jugador()
+anotarGol()
+jugarPartido(int)
+recibirAmarilla()
+recibirRoja()
+hacerFalta()
+mostrar()
+getGoles() : int
+getNumero() : int

C. Describa en alto nivel la lógica de las tareas que usted definió para aquellos subprogramas cuya solución no sea trivial. No copie y pegue código en este apartado.

La implementación de solución se hace de manera modular mediante el uso de headers .h que contienen clases que garantizan que los datos solo se modifiquen a través de métodos seguros (encapsulamiento), manteniendo la integridad de las estadísticas incluso en procesos complejos como el de los Mejores Terceros, y archivos .cpp que contienen la lógica del programa, el desarrollo de funciones, constructores, guardado de datos y estadísticas, impresión de datos y trabajo conjunto con otros archivos.

Simulación de encuentros:

La solución no está dada por un simple azar, sino un algoritmo de búsqueda que permite que se organicen los equipos en 4 contenedores denominados bombos según su ranking FIFA, en este caso para cada grupo desde A hasta el L se extrae un equipo de cada bombo, posteriormente se hace la validación de restricciones, antes de asignar un equipo, el algoritmo verifica la confederación de los integrantes ya existentes en el grupo, si se viola la regla por ejemplo dos equipos de CONMEBOL en el mismo grupo o más de dos de la UEFA, el algoritmo descarta la selección actual, devuelve el equipo al bombo y repite el proceso hasta hallar una combinación válida que respete la geografía del torneo.

Para la simulación de los encuentros se hace uso del modelo de Poisson y eventos discretos, fórmulas planteadas desde la guía donde la lógica de los partidos se divide en dos fases, el cálculo de probabilidad utilizando la ecuación de poisson para determinar el parámetro lambda que hace referencia a los goles esperados. El segundo es simulación por probabilidad acumulada donde una vez definido el número de goles, el programa itera sobre los 11 jugadores convocados, allí mediante un generador de números pseudoaleatorios y umbrales de probabilidad predefinidos como el 4% para goles, 6% para amarillas, etc, se asignan los eventos a jugadores específicos hasta agotar la cuota de goles del equipo o finalizar el tiempo reglamentario.

Selección de los mejores terceros

Este subprograma actúa como un filtro de ordenamiento global sobre ciertos subconjuntos donde al finalizar la fase de grupos, se extrae el equipo posicionado en el tercer lugar de cada uno de los 12 grupos y se insertan en una estructura temporal,

Se aplica un algoritmo de ordenamiento descendente basado en una jerarquía de criterios: primero Puntos, luego Diferencia de Goles, seguido de Goles a Favor y, finalmente, un sorteo aleatorio en caso de paridad absoluta. Los primeros 8 equipos de este ranking se marcan como clasificados y se distribuyen en las llaves de Dieciseisavos (R16) evitando que se enfrenten a equipos que pertenecieron a su mismo grupo original.

Problemas de desarrollo que afrontó.

Si tuviéramos que contar qué fue lo más difícil de este proyecto, diríamos que hubo retos en la comprensión del funcionamiento del fútbol, en realidad no teníamos mucha claridad sobre ello, en especial el trabajo no fue pensado para las personas que no saben ni gustan en lo absoluto del deporte, además la lucha contra la memoria y las restricciones impuestas por el docente.

El primer gran dolor de cabeza fue gestionar todo a mano al no poder usar las herramientas automáticas de C++, si cometíamos algún error con un puntero o se nos olvidaba liberar un pedacito de memoria, el programa colapsaba o perdía información de los jugadores.

La lógica general y específica de cada parte del programa fue muy demandante, en especial por el desconocimiento del deporte, pero también por la correcta implementación que respondiera a todas las demandas, digamos que habían ciertos detalles específicos como no poder jugar más de 4 partidos y que los equipos que jugaran debían descansar 3 días que demandó una reinterpretación de la lógica y del código en varias ocasiones, además fue muy frustrante cuando el programa presentaba muchos errores o incluso funcionando, la guía pedía cosas diferentes o más detalladas que nos obligaban a cambiar e implementar muchas otras cosas.

Luego vino el rompecabezas del sorteo, no era solo tirar los equipos al azar, tenía que seguir las reglas de la FIFA que dicen que no pueden haber muchos equipos de la misma confederación en un grupo. Fue como intentar acomodar a un grupo de amigos en mesas de una fiesta, pero con la condición de que ciertos grupos no se pueden ni ver, y otros solo se aguantan de a dos. Lograr que el algoritmo encontrara esa combinación perfecta sin quedarse dando vueltas en un bucle infinito fue un reto de lógica bastante fuerte.

Otro momento crítico fue la organización de los mejores terceros. Organizar a esos equipos fue difícil porque nunca jugaron entre ellos. Tuvimos que programar una jerarquía de decisiones: si empatan en puntos, miren los goles; si empatan en goles, miren la diferencia; y si todo sigue igual, hagan un sorteo. Hacer que el código tomara esas decisiones con total imparcialidad y sin errores fue muy exigente.

También estuvo el tema del calendario y el descanso, creemos que fue uno de los más complicados, controlar que ningún equipo jugara cansado respetando los 3 días de descanso y que no se saturara la sede con más de 4 partidos al día, nos obligó a ser muy ordenados con las fechas.

Evolución de la solución y consideraciones a tener en cuenta en la implementación.

La construcción de nuestro sistema UdeAWorldCup 2026 no fue un proceso lineal, sino una evolución marcada por la necesidad de optimizar recursos y garantizar la integridad de los datos bajo las restricciones de la guía, en este caso el UML preliminar fue de gran ayuda ya que nos permitió una organización inicial modular del programa en QT, estableciendo grupos de trabajo. Inicialmente la evolución del diseño del programa parte de una fase inicial de análisis y ordenamiento, el proyecto comenzó por definir las estructuras más básicas y fundamentales para el funcionamiento del programa, definiendo las clases iniciales que incluían a los

jugadores y los equipos, desarrollando la lógica inicial del programa por supuesto pensando en la futura integración con los demás módulos para el funcionamiento global del programa, allí definimos clases, objetos, atributos, constructores y desarrollo de funciones básicas que funcionarían por debajo del programa y serían utilizadas más adelante.

Hubo una fase intermedia donde adoptamos plantillas y memoria dinámica, e implementamos funciones más complejas como lo son grupo y equipo, todo lo hicimos pensando inicialmente en el correcto funcionamiento sin tener en cuenta de manera estricta las restricciones propuestas por el profesor. Ante la prohibición de la STL, la solución evolucionó hacia la implementación de funciones específicas, operaciones matemáticas y una integración detallada de las mismas, en este punto teníamos un gran avance del trabajo, pero aún presentaba algunos errores y no cumplía con los requerimientos de la guía.

Para la fase final hicimos el refinamiento, nos atrevemos a decir que fue una de las más complejas. La última etapa se centró en la inteligencia del torneo, pulir errores y asegurar el correcto funcionamiento, además de la integración del medidor de uso de memoria. Pasamos de un sorteo aleatorio a uno con restricciones geográficas y refinamos el sistema de desempate de los mejores terceros, asegurando que el flujo del programa fuera un reflejo fiel de un mundial real. Tuvimos en cuenta las fechas, se organizarían correctamente las impresiones de tablas y el guardado de archivo al final del programa. Luego de una revisión detallada y el visto bueno de ambos integrantes dimos por finalizado el proyecto.

Al principio planteamos una estructura básica, pero las exigencias de la guía como evitar que se repitan confederaciones en los grupos nos obligaron a desarrollar funciones mucho más lógicas y detalladas.

Dentro de las consideraciones clave que tuvimos en cuenta estuvieron la conexión entre clases, dado que decidimos que cada clase equipo fuera la encargada de almacenar y controlar su propia lista de 26 Jugadores. Esta forma de organizar los datos nos permitió que, al ocurrir un evento en un partido como un gol o una tarjeta, la información se actualizara directamente en el jugador correspondiente sin enredar el resto del programa.

Hicimos la gestión manual de la memoria, diseñamos nuestras propias estructuras para manejar los datos. Nos aseguramos en la medida de lo posible de pedir espacio en la memoria solo cuando era estrictamente necesario y de liberarlo manualmente apenas dejaba de usarse. Esto fue fundamental para cumplir con el requisito de eficiencia y evitar que el programa se volviera pesado.

Para que los resultados fueran realistas, aplicamos la fórmula de Poisson sugerida en la guía. Esto permitió que el marcador de cada partido dependiera del Ranking FIFA de los equipos involucrados, logrando que la simulación fuera coherente con el nivel deportivo de cada país y no un resultado puramente al azar.

Al final en cada etapa del código incluimos contadores para rastrear cuántas instrucciones ejecutaba el procesador.